

Lecture 12

Hash Tables

15-122: Principles of Imperative Computation (Spring 2025)
Frank Pfenning, Rob Simmons

Dictionaries, also called *associative arrays* as well as *maps*, are data structures that are similar to arrays but are not indexed by integers, but by possibly other forms of data such as strings. One popular data structure for the implementation of dictionaries are *hash tables*. To analyze the asymptotic efficiency of hash tables we have to explore a new point of view, that of *average case complexity*. Another computational thinking concept that we revisit is *randomness*. In order for hash tables to work efficiently in practice we need hash functions whose behavior is predictable (deterministic) but has some aspects of randomness.

Additional Resources

- Review slides
 - [Generic Pointers](https://cs.cmu.edu/~15122/handouts/slides/review/12-voidstar.pdf) (<https://cs.cmu.edu/~15122/handouts/slides/review/12-voidstar.pdf>)
 - [Hashing](https://cs.cmu.edu/~15122/handouts/slides/review/12-hashing.pdf) (<https://cs.cmu.edu/~15122/handouts/slides/review/12-hashing.pdf>)
- [Code for this lecture](https://cs.cmu.edu/~15122/handouts/code/12-hashing.tgz) (<https://cs.cmu.edu/~15122/handouts/code/12-hashing.tgz>)
- There is one short video associated with this lecture:
 - [Hash Tables](https://youtu.be/TngAkEAAA5s) (<https://youtu.be/TngAkEAAA5s>)

Relating to our learning goals, we have

Computational Thinking: We consider the importance of *randomness* in algorithms, and also discuss *average case analysis*, which is how we can argue that hash tables have acceptable performance.

Algorithms and Data Structures: We describe a *linear congruential generator*, which is a certain kind of *pseudorandom number generator*. We also discuss hash tables and their implementation with *separate chaining* (an array of linked lists).

Programming: We review the implementation of the `rand` library in C0.

1 Dictionaries, Associative Arrays, or Maps

An array A can be seen as a mapping $i \mapsto v$ that associates a value v (which we denote $A[i]$) with every index i in the range $[0, \backslash length(A))$. It is finitary, because its domain, and therefore also its range, is finite. There are many situations when we want to index elements differently than just by contiguous integers starting at 0. Common examples are strings (for dictionaries, phone books, menus, database records), or structs (for dates, or names together with other identifying information). Such generalized arrays are called *dictionaries* or *associative arrays* or *maps*. These situations are so common that dictionaries are primitive in some languages such as PHP, Python, or Perl and perhaps account for some of the popularity of these languages. In many applications, dictionaries are implemented as hash tables because of their performance characteristics. We will develop them incrementally to understand the motivation underlying their design.

2 Keys, Entries and Values

In many applications requiring dictionaries, we are storing complex data and want to access them by a *key* which is derived from the data. For example, the key might be a student id (a string) and the data might be this student's record, which may itself record her grades in a dictionary whose keys are the name of each assignment or exam and the data is a score. We make the assumption that keys are unique in the sense that in a dictionary there is at most one data item associated with a given key — here no two students have the same id and no two assignments have the same name.

The data item associated to a key in a dictionary is variously called an *entry* or a *value*. We tend to use the word *entry* when the key is part of the data (like a student record) and the word *value* when it is not (for example, the average temperature of each day of the year).

We can think of built-in C0 arrays as dictionaries having a set number of keys: a C0 array of length 3 has three keys 0, 1, and 2. Our implementation of unbounded arrays allowed us to add a specific new key, 3, to such an array. We do want to be able to add new keys to the dictionary. We also want our dictionaries to allow us to have more interesting keys (like strings, or non-sequential integers) while keeping the property that there is a unique entry for each valid key.

3 Chains

A first idea to explore is to implement the dictionary as a linked list, called a *chain*. If we have a key k and look for it in the chain, we just traverse it, compute the intrinsic key for each data entry, and compare it with k . If they are equal, we have found our entry, if not we continue the search. If we reach the end of the chain and do not find an entry with key k , then no entry with the given key exists. If we keep the chain unsorted this gives us $O(n)$ worst case complexity for finding a key in a chain of length n , assuming that computing and comparing keys is constant time.

Given what we have seen so far in our search data structures, this seems very poor behavior, but if we know our data collections will always be small, it may in fact be reasonable on occasion.

Can we do better? One idea goes back to binary search. If keys are ordered we may be able to arrange the elements in an array or in the form of a tree and then cut the search space roughly in half every time we make a comparison. Designing such data structures is a rich and interesting subject, but the best we can hope for with this approach is $O(\log n)$, where n is the number of entries. We have seen that this function grows very slowly, so this is quite a practical approach.

Nevertheless, the challenge arises if we can do better than $O(\log n)$, say, constant time $O(1)$ to find an entry with a given key. We know that it can be done for arrays, indexed by integers, which allow constant-time access. Can we also do it, for example, for strings?

4 Hashing

The first idea behind hash tables is to exploit the efficiency of arrays. So: to map a key to an entry, we first map a key to an integer and then use the integer to index an array A . The first map is called a *hash function*. We write it as $\text{hash}(_)$. Given a key k , our access could then simply be $A[\text{hash}(k)]$.

There is an immediate problem with this approach: there are 2^{31} positive integers, so we would need a huge array, negating any possible performance advantages. But even if we were willing to allocate such a huge array, there are many more strings than **int**'s so there cannot be any hash function that always gives us different **int**'s for different strings.

The solution is to allocate an array of smaller size, say m , and then look up the result of the hash function modulo m , for example, $A[\text{hash}(k) \% m]$. This idea has an obvious problem: it is inevitable that multiple strings will map to the same array index. For example, if the array has size m then if we have more than m elements, at least two must map to the same index

— this simple observation is an instance of what is known as the *pigeonhole principle*. In practice, this will happen much sooner than this.

If a hash function maps two keys to the same integer value (modulo m), we say we have a *collision*. In general, we would like to avoid collisions, because some additional operations will be required to deal with them, slowing down search and taking more space. We analyze the cost of collisions more below.

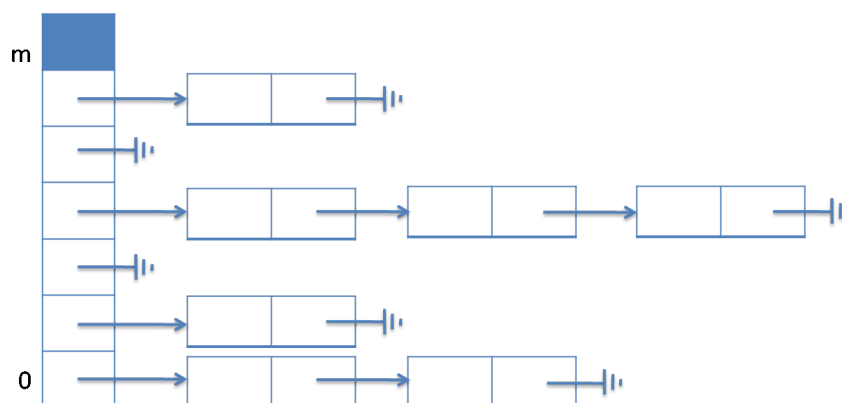
5 Separate Chaining

How do we deal with collisions of hash values? The simplest is a technique called *separate chaining*. Assume we have $\text{hash}(k_1) \% m = i = \text{hash}(k_2) \% m$, where k_1 and k_2 are the distinct keys for two data entries e_1 and e_2 we want to store in the table. In this case we just arrange e_1 and e_2 into a chain (implemented as a linked list) and store this list in $A[i]$.

In general, each element $A[i]$ in the array will either be NULL or a chain of entries. All of these must have the same hash value for their key (modulo m), namely i . As an exercise, you might consider other data structures here instead of chains and weigh their merits: how about sorted lists? Or queues? Or doubly-linked lists? Or another hash table?

We stick with chains because they are simple and fast, provided the chains don't become too long. This technique is called *separate chaining* because the chains are stored separately, not directly in the array. Another technique, which we will not discuss at length, is *linear probing* where we continue by searching (linearly) for an unused spot in the array itself, starting from the place where the hash function put us.

Under separate chaining, a snapshot of a hash table might look something like this picture.



6 Average Case Analysis

See the short video on [Hash Tables](https://www.youtube.com/embed/TngAkEAAA5s) at <https://www.youtube.com/embed/TngAkEAAA5s>.

How long do we expect the chains to be on average? For a total number n of entries in a table of size m , it is n/m . This important number is also called the *load factor* of the hash table. How long does it take to search for an entry with key k ? We follow these steps:

1. Compute $i = \text{hash}(k) \% m$. This will be $O(1)$ (constant time), assuming it takes constant time to compute the hash function.
2. Go to $A[i]$, which again is constant time $O(1)$.
3. Search the chain starting at $A[i]$ for an element whose key matches k . We will analyze this next.

The complexity of the last step depends on the length of the chain. In the *worst case* it could be $O(n)$, because all n elements could be stored in one chain. This worst case could arise if we allocated a very small array (say, $m = 1$), or because the hash function maps all input strings to the same table index i , or just out of sheer bad luck.

Ideally, all the chains would be approximately the same length, namely n/m . Then for a fixed load factor such as $n/m = 2.0$ we would take on average 2 steps to go down the chain and find k . In general, as long as we don't let the load factor become too large, the *average* time should be $O(1)$.

If the load factor does become too large, we could dynamically adapt the size of the array, like in an unbounded array. As for unbounded arrays, it is beneficial to double the size of the hash table when the load factor becomes too high, or possibly halve it if the size becomes too small. Analyzing these factors is a task for amortized analysis, just as for unbounded arrays. If we do so, insertion would have an amortized average time complexity of $O(1)$ — *amortized* because the table needs to be resized from time to time and *average* if we manage to have all the chains be about the same length. Searching for a key would cost $O(1)$ on average for the same reason — not amortized though because search never triggers a resize.

7 Randomness

The average case analysis relies on the fact that the hash values of the key are relatively evenly distributed. This can be restated as saying that the probability that each key maps to an array index i should be about the same, namely $1/m$. In order to avoid systematically creating collisions,

small changes in the input string should result in unpredictable change in the output hash value that is uniformly distributed over the range of C0 integers. We can achieve this with a *pseudorandom number generator* (PRNG). A pseudorandom number generator is just a function that takes one number and obtains another in a way that is both unpredictable and easy to calculate. The C0 rand library is a pseudorandom number generator with a fairly simple interface:

```
/* library file rand.h0 */
typedef struct rand* rand_t;
rand_t init_rand (int seed);
int rand(rand_t gen);
```

One can generate a random number generator (type `rand_t`) by initializing it with an arbitrary seed. Then we can generate a sequence of random numbers by repeatedly calling `rand` on such a generator.

The `rand` library in C0 is implemented as a *linear congruential generator*. A linear congruential generator takes a number x and finds the next number by calculating $(a \times x) + c$ modulo d , a number that is used as the next x . In C0, it's easiest to say that d is just 2^{32} , since addition and multiplication in C0 are already defined modulo 2^{32} . The trick is finding a good multiplier a and summand c .

If we were using 4-bit numbers (from -8 to 7 where multiplication and addition are modulo 16) then we could set a to 5 and c to 7 and our pseudorandom number generator would generate the following series of numbers:

$$0 \rightarrow 7 \rightarrow (-6) \rightarrow (-7) \rightarrow 4 \rightarrow (-5) \rightarrow (-2) \rightarrow \\ -3 \rightarrow (-8) \rightarrow (-1) \rightarrow 1 \rightarrow (-4) \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 0 \rightarrow \dots$$

The PRNG used in C0's library sets a to 1664525 and c to 1013904223 and generates the following series of numbers starting from 0:

$$0 \rightarrow 1013904223 \rightarrow 1196435762 \rightarrow (-775096599) \rightarrow (-1426500812) \rightarrow \dots$$

This kind of generator is fine for random testing or (indeed) the basis for a hashing function, but the results are too predictable to use it for cryptographic purposes such as encrypting a message. In particular, a linear congruential generator will sometimes have repeating patterns in the lower bits. If one wants numbers from a small range it is better to use the higher bits of the generated results rather than just applying the modulus operation.

It is important to realize that these numbers just *look* random, they aren't really random. In particular, we can reproduce the exact same sequence if

we give it the exact same seed. This property is important for both testing purposes and for hashing. If we discover a bug during testing with pseudorandom numbers, we want to be able to reliably reproduce it, and whenever we hash the same key using pseudorandom numbers, we need to be sure we will get the same result.

```
1 /* library file rand.c0 */
2 struct rand {
3     int seed;
4 };
5
6 rand_t init_rand (int seed) {
7     rand_t gen = alloc(struct rand);
8     gen->seed = seed;
9     return gen;
10 }
11
12 int rand(rand_t gen) {
13     gen->seed = gen->seed * 1664525 + 1013904223;
14     return gen->seed;
15 }
```

Observe that some choices of the numbers a and c would be terrible. For example, if we were to work with 4-bit numbers, so that $d = 16$, choosing $a = 0$ would mean that our “pseudo-random” generator always returns c . Were we to choose $a = c = 4$, it would only return values among -8 , -4 , 0 , and 4 . In general, we want, at a minimum, that the factor c and the modulus d be *relatively prime*, i.e., that their greatest common divisor be 1.

8 Exercises

Exercise 1 (sample solution on page 9). In a separate-chaining hash dictionary, what happens when you implement buckets with some data structure other than a linked list? Discuss the changes and identify benefits and disadvantages when using the data structures seen so far in this course (e.g., an array, a sorted list, a queue, a stack, or another hash table for separate chaining).

Exercise 2 (sample solution on page 10). We are writing a hash function for strings of length exactly two, and they consist of only the characters 'A' to 'Z'. There are $26 \times 26 = 676$ different such strings. Your application only uses 79 out of those 676 two-letter words. So, you were hoping to implement a hash function that does not cause collisions on your string. But you still see collisions most of the time. Look up the birthday paradox and use it to explain this phenomenon.

Exercise 3 (sample solution on page 10). Assume you have a separate-chaining hash dictionary of size 10, which stores keys of type `int`.

1. Given a hash function: $\text{hash}(x) = 2x + 1$ and a sequence of values: 8, 122, 127, 42, 13, 15. If we insert this sequence of values into the hash table in order:
 - How many chains are non-NULL (have values inserted into it)?
 - What's the size of the longest chain? What values are stored in it?
2. Give a hash function such that the chain at index 9 contains the following values: 16, 37, 56, 17, 36

Sample Solutions

Solution of exercise 1

Let's go through the data structures we know about, besides linked lists. Throughout the following discussion, we will assume the hash dictionary contains n entries. We will consider the worst case, where the table is not self-resizing and the hash function may not be very good. We know that, in this setup, implementing buckets using linked lists yields an $O(n)$ cost for both looking up a key and inserting an entry.

Arrays: An array wouldn't do us much good since arrays have fixed size. This would not be a viable bucket implementation.

Unbounded arrays: On the other hand, an unbounded array can grow as needed. Looking up a key would rely on linear search and cost $O(n)$. Although a new entry could simply be added at the end of the unbounded array, at a cost of $O(1)$ average and amortized, we first need to make sure it is not in the unbounded array. This bumps up the cost to $O(n)$.

But we can do better if we keep the unbounded array sorted! Then, we can use binary search for both operations, which lower the cost to $O(\log n)$.

Sorted lists: Maintaining our buckets sorted by keys does not help much. We would still need to perform linear search to lookup a key or insert an entry, which causes the worst-case complexity to be $O(n)$. In practice, sorted buckets enable to stop the search early if we encounter a larger key than the one we are looking for or trying to insert. This would not affect the asymptotic complexity of the search.

Queues: When we look up a key, in the worst case all n entries will be in one queue and we would have to dequeue everything off the queue to find it. Thus, the worst case lookup costs $O(n)$. Inserting an entry would have similar cost. The runtime of using a queue isn't very different from that of the linked list.

Stacks: The analysis we performed for queues applied identically to stacks.

Hash tables: When looking up a key in a bucket, a good hash table implementation of buckets would return the associated entry in time $O(1)$ average, with insertion being $O(1)$ average and amortized. If the hash table implementation of buckets is not too good, we may run into the same problem of having all the entries in one chain of

the hash table implementing the bucket, thus the worst case scenario would be $O(n)$.

Based on this analysis, our best bet is to either use a sorted unbounded array or, even better, a good hash table. However, if the hash dictionary employs a good hash function and resizes the underlying table when the load factor reaches a constant value (e.g., 1.0), then the expected performance will be indistinguishable from using a linked list to implement buckets. And linked lists are a lot easier to implement than those data structures.

Solution of exercise 2

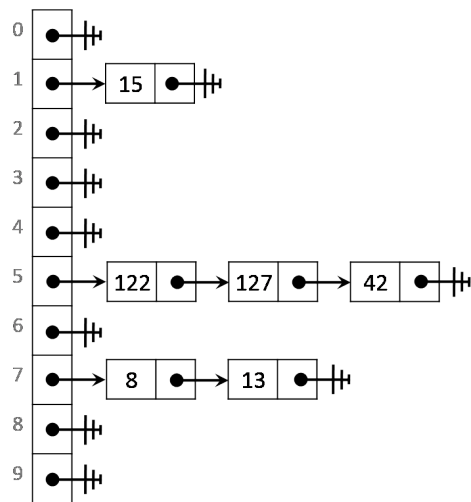
First of all, you will have collision if your table size is any less than 79. This is by a different mathematical law: the pigeon-hole principle.

Knowing this, you picked a table size greater than 79, but less than 676 since that would be wasting a lot of space in your mind. Let's say your table has length 100. The villain this time is the *birthday paradox*, which is maps out the probability that two people have the same birthday in a group of n randomly chosen individuals. For example, it takes only 23 people to get a 50% probability that two of them will have the same birthday. This probability jumps to 90% with 70 people.

The birthday paradox applied to collisions in hash tables as follows. The number of people sharing a birthday corresponds to the number of entries we insert in the hash table. The capacity of the table replaces the number of days in a year, 365. With this in mind, the birthday paradox tells us that there's a high probability that two of our 79 keys will collide given a smallish table with 100 positions. Of course, given a reasonable hash function, the larger the table size the less likely it is to have a collision. It cannot be ruled out however, unless the table has more position than the possible number of keys (here 676).

Solution of exercise 3

After this sequence of insertions, the resulting hash table is as follows (for simplicity, we assume entries are inserted at the end of a chain):



Therefore:

- There are 3 non-NULL chains.
 - The longest chain, at table index 5, has size 3, and it contains the value 122, 127 and 42.
2. There are many possible answers of course, the simplest being $hash(x) = 9$, which would be a horrible has function. A better function would be $hash(x) = x/2 + 1$.